VagaCerta - Relatório de implementação

Persistência local, armazenamento seguro de credenciais, autenticação e recursos nativos

Responsável: Lucas Sehn Klauck | Expo SDK 58 / React Native 0.88 / Expo Router / Clerk

As tarefas do projeto foram divididas entre os integrantes do grupo. Estas funcionalidades ficaram sob responsabilidade exclusiva de Lucas; os demais integrantes têm as suas próprias responsabilidades no projeto. Assim, conforme a divisão pedida pelo professor, coube a mim realizar somente estas tarefas.

1. Persistência local e armazenamento seguro

Nada sensível fica em texto puro. Tudo que persiste no aparelho usa o **expo-secure-store** (Keychain no iOS, Keystore no Android); no navegador, onde o SecureStore não existe, cai para **localStorage**.

O que é salvo	Onde / como	Arquivo
Token e sessão do Clerk	tokenCache oficial do Clerk, que usa SecureStore	src/app/_layout.tsx
Onboarding concluído (onboarding_done)	SecureStore, helpers read/write	src/lib/onboarding.ts
Lembrar de mim (remember_me)	SecureStore, helpers read/write	src/lib/onboarding.ts
Preferências dos switches de notificação	SecureStore, JSON (notification_prefs)	src/lib/onboarding.ts
Localização do usuário	NÃO é salva: fica só em memória (RNF06)	src/hooks/use-user-location.ts
Segredos (chaves)	Variáveis EXPO_PUBLIC_* no .env, ignorado pelo Git	.env / .gitignore

Como foi feito

src/lib/onboarding.ts concentra a leitura e a escrita em duas funções, *read* e *write*, com try/catch para o app nunca quebrar se o armazenamento falhar. As demais funções são uma linha cada:

```
function read(key: string) { /* SecureStore.getItem ou localStorage.getItem, com try/catch */ }
function write(key: string, value: string) { /* SecureStore.setItem ou localStorage.setItem */ }

export const hasOnboarded   = () => read('onboarding_done') === '1';
export const markOnboarded  = () => write('onboarding_done', '1');
export const shouldRemember = () => read('remember_me') !== '0';
export const setRemember    = (v: boolean) => write('remember_me', v ? '1' : '0');
```

O *shouldRemember* compara com **!== '0'** (e não === '1') para que o padrão, sem nada salvo, seja lembrar. Assim só desloga quem desmarcou de propósito.

2. Autenticação (Clerk)

Tela / arquivo	O que faz
src/app/_layout.tsx	ClerkProvider com a publishable key e o tokenCache seguro. Proteção de rotas com Stack.Protected: deslogado vê login/cadastro/esqueci-senha; logado vê o onboarding (se nunca viu) ou a Home.
src/app/index.tsx (Login)	E-mail e senha ligados ao Clerk, estado de carregando no botão, erros amigáveis, código por e-mail (device trust/2FA), Google e Apple via useSSO, links para Criar conta e Esqueci minha senha, "Lembrar de mim".
src/app/cadastro.tsx	Nome, e-mail, senha e confirmação com validação, verificação por código enviado ao e-mail, Google e Apple, link "Já tenho conta".

Tela / arquivo	O que faz
src/app/esqueci-senha.tsx	Passo 1: e-mail. Passo 2: código e nova senha. Volta ao login com mensagem de sucesso.
src/lib/api.ts	fetch autenticado: anexa o token do Clerk (Authorization: Bearer) e usa EXPO_PUBLIC_API_URL.
src/lib/clerk-error.ts	Traduz os erros do Clerk para mensagens em português.

Lembrar de mim (funcionalidade)

O Clerk sempre persiste a sessão. Então "lembrar" funciona ao contrário: se o usuário desmarcar, a sessão é encerrada na próxima abertura do app. Em três pontos:

• **Login (index.tsx):** a função run(), por onde passam todos os logins (senha, código, Google, Apple), chama saveRemember(remember) antes de cada tentativa.

• **Abertura (_layout.tsx):** um efeito com useRef roda uma única vez por abertura; se há sessão e shouldRemember() é false, chama signOut(). O useRef evita deslogar logo após o login.

• **Redirecionamento:** um segundo efeito leva o usuário logado para /localizacao (primeira vez) ou /home, evitando a rota "unmatched" em "/".

```
const checked = useRef(false);
useEffect(() => {
  if (!isLoaded || checked.current) return;
  checked.current = true;
  if (isSignedIn && !shouldRemember()) signOut();
}, [isLoaded, isSignedIn, signOut]);

useEffect(() => {
  if (isLoaded && isSignedIn) router.replace(hasOnboarded() ? '/home' : '/localizacao');
}, [isLoaded, isSignedIn]);
```

3. Recursos nativos (permissões)

Fluxo de onboarding: Localização -> Notificações -> Microfone -> Home. Em todas as telas, aceitar ou negar segue o fluxo; nada trava. Pacotes instalados com *npx expo install*: expo-location, expo-notifications, expo-audio.

3.1 Localização (RF01) - src/app/localizacao.tsx e src/hooks/use-user-location.ts

• "Permitir acesso à localização" chama Location.requestForegroundPermissionsAsync() e depois navega para Notificações, aceitando ou negando. "Agora não" não pede nada.

• Hook compartilhado **useUserLocation()** devolve { status, coords, request }. Usa Location.useForegroundPermissions e busca a posição só quando a permissão é concedida. Se o GPS falhar ou a permissão for negada, coords fica null e a Home funciona sem a posição.

• As coordenadas ficam só em estado do React (memória), nunca em disco (RNF06).

3.2 Notificações - src/app/notificacoes.tsx

• Só a solicitação de permissão (push remoto está fora do escopo). No Android 13+ o prompt só aparece depois de existir um canal, por isso a tela cria o canal "default" antes de chamar requestPermissionsAsync().

• Os três switches (vagas, lembretes, ofertas) iniciam com getNotificationPrefs() e são salvos com setNotificationPrefs() ao tocar em "Permitir" (SecureStore, JSON), para o Perfil poder ler e alterar.

• No Expo Go do Android o push remoto não é suportado desde o SDK 53; a permissão local funciona.

3.3 Microfone (RF17) - src/app/microfone.tsx e src/hooks/use-mic-permission.ts

• Hook compartilhado **useMicPermission()** devolve { status: undetermined | granted | denied, canAskAgain, request() }, usando AudioModule do expo-audio (funciona no Expo Go).

• "Permitir" chama request() e segue para a Home, aceitando ou negando; a pesquisa por texto continua disponível.

• Se a permissão está negada de forma permanente (denied e canAskAgain falso), o botão vira "Abrir configurações" e chama Linking.openSettings().

• O texto da tela diz que o microfone é usado só para pesquisar endereços por voz.

3.4 Mensagens de uso (app.json)

Plugin	Configuração
expo-location	locationWhenInUsePermission: usa a localização para mostrar estacionamentos e vagas perto do usuário
expo-audio	microphonePermission: usa o microfone apenas para pesquisar endereços por voz; enableBackgroundPlayback: false
expo-notifications	plugin padrão (iOS não exige texto de uso para notificações)

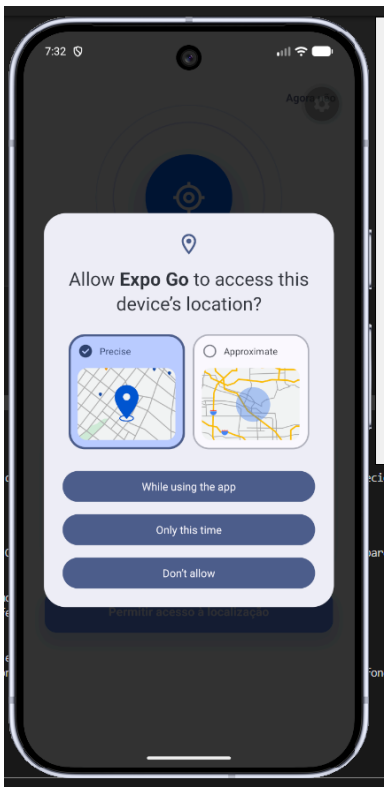
Essas mensagens só aparecem em build nativo (development build / EAS); no Expo Go vale o texto padrão do próprio app.

4. Evidências

Capturas de tela do fluxo de permissões no Expo Go (emulador Android). Cada tela de permissão é seguida do aviso nativo do sistema, que é a permissão real solicitada ao usuário.



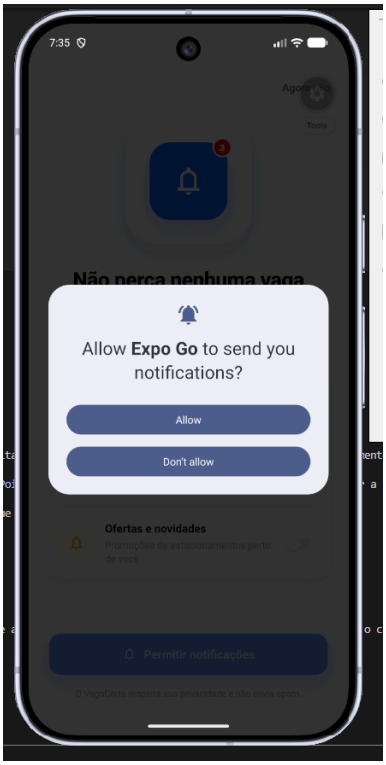
Tela de Localização



Aviso do sistema: localização



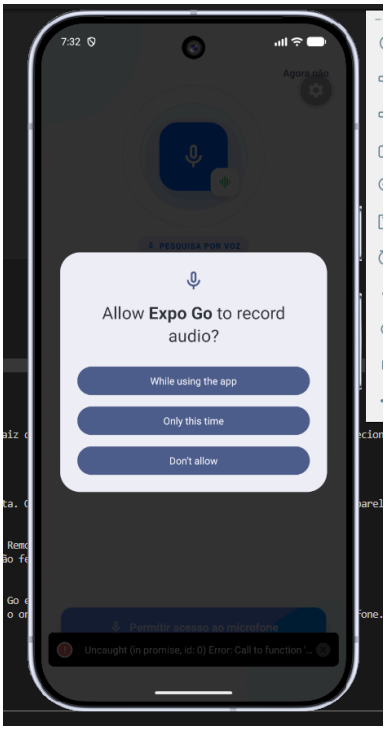
Tela de Notificações



Aviso do sistema: notificações



Tela de Microfone



Aviso do sistema: microfone